Software Project Management 4th Edition

Chapter 5

Software effort estimation

What makes a successful project?

Delivering:

- agreed functionality
- on time
- at the agreed cost
- with the required quality

Stages:

1. set targets
2. Attempt to achieve targets

BUT what if the targets are not achievable?

Over and under-estimating

- Parkinson's Law: 'Work expands to fill the time available'
- An over-estimate is likely to cause project to take longer than it would otherwise
- Weinberg's Zeroth Law of reliability: 'a software project that does not have to meet a reliability requirement can meet any other requirement'

A taxonomy of estimating methods

- Bottom-up - activity based, analytical
- Parametric or algorithmic models e.g. function points
- Expert opinion - just guessing?
- Analogy - case-based, comparative
- Parkinson and 'price to win'

Some estimation approach (wiki)

- Analogy-based estimation - Formal estimation model
- WBS-based (bottom up) estimation - Expert estimation
- Parametric models - Formal estimation model
  - COCOMO, SLIM, SEER-SEM, TruePlanning for Software
- Size-based estimation models- Formal estimation model
  - Function Point Analysis,[14] Use Case Analysis, SSU (Software Size Unit), Story points-based estimation in Agile software development
- Group estimation - Expert estimation
  - Planning poker, Wideband Delphi
- Mechanical combination - Combination-based estimation
- Judgmental combination - Combination-based estimation
  - Expert judgment based on estimates from a parametric model and group estimation

Bottom-up versus top-down

- Bottom-up
  - use when no past project data
  - identify all tasks that have to be done – so quite time-consuming
  - use when you have no data about similar past projects
- Top-down
  - produce overall estimate based on project cost drivers
  - based on past project data
  - divide overall estimate between jobs to be done

Bottom-up estimating

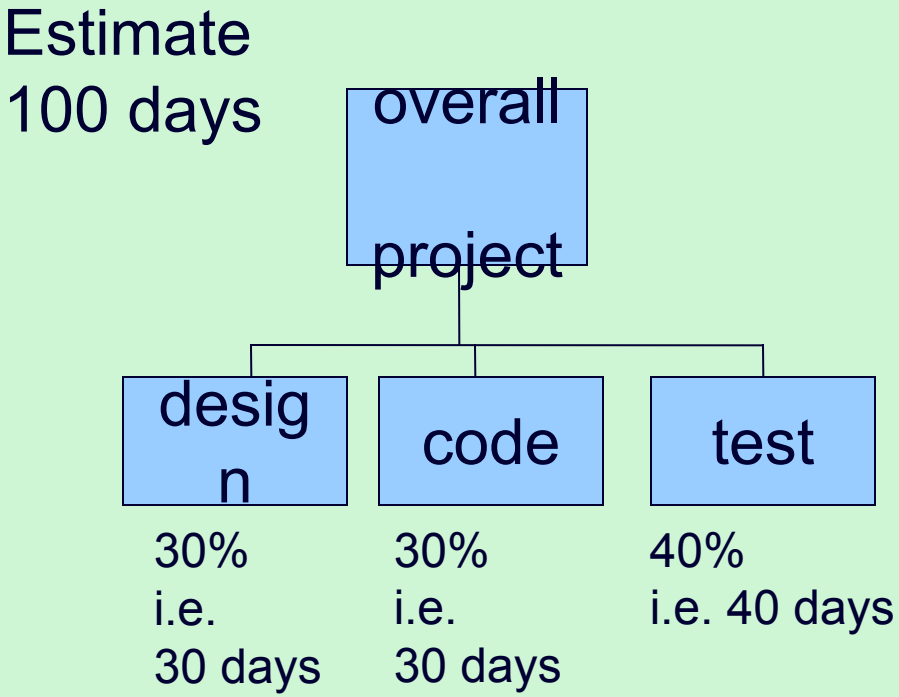
1. Break project into smaller and smaller components

[2. Stop when you get to what one person can do in one/two weeks]

3. Estimate costs for the lowest level activities

4. At each higher level calculate estimate by adding estimates for lower levels

Top-down estimates



- Produce overall estimate using effort driver (s)
- distribute proportions of overall estimate to components

Algorithmic/Parametric models

- COCOMO (lines of code) and function points examples of these
- Problem with COCOMO etc:

but what is desired is

Parametric models - continued

- Examples of system characteristics
  - no of screens x 4 hours
  - no of reports x 2 days
  - no of entity types x 2 days
- the quantitative relationship between the input and output products of a process can be used as the basis of a parametric model

Parametric models - the need for historical data

- simplistic model for an estimate

  estimated effort = (system size) / productivity

  e.g.

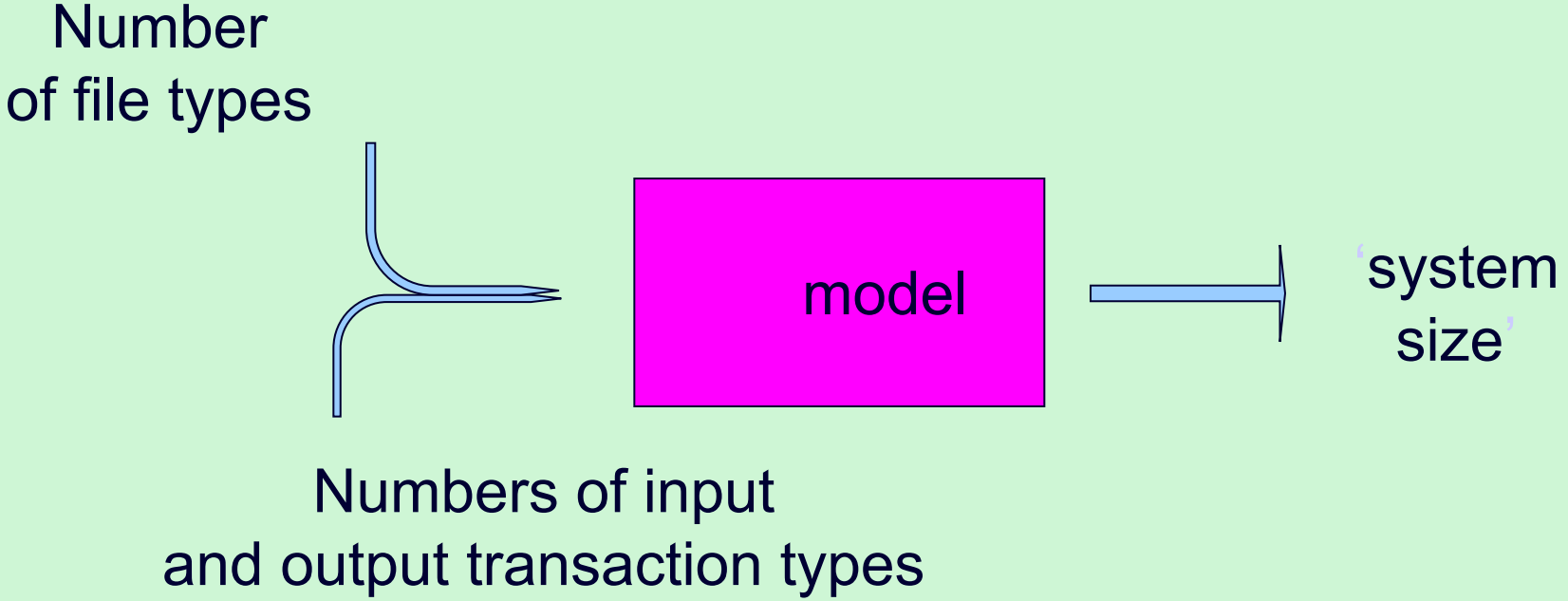
  system size = lines of code

  productivity = lines of code per day

- productivity = (system size) / effort
  - based on past projects

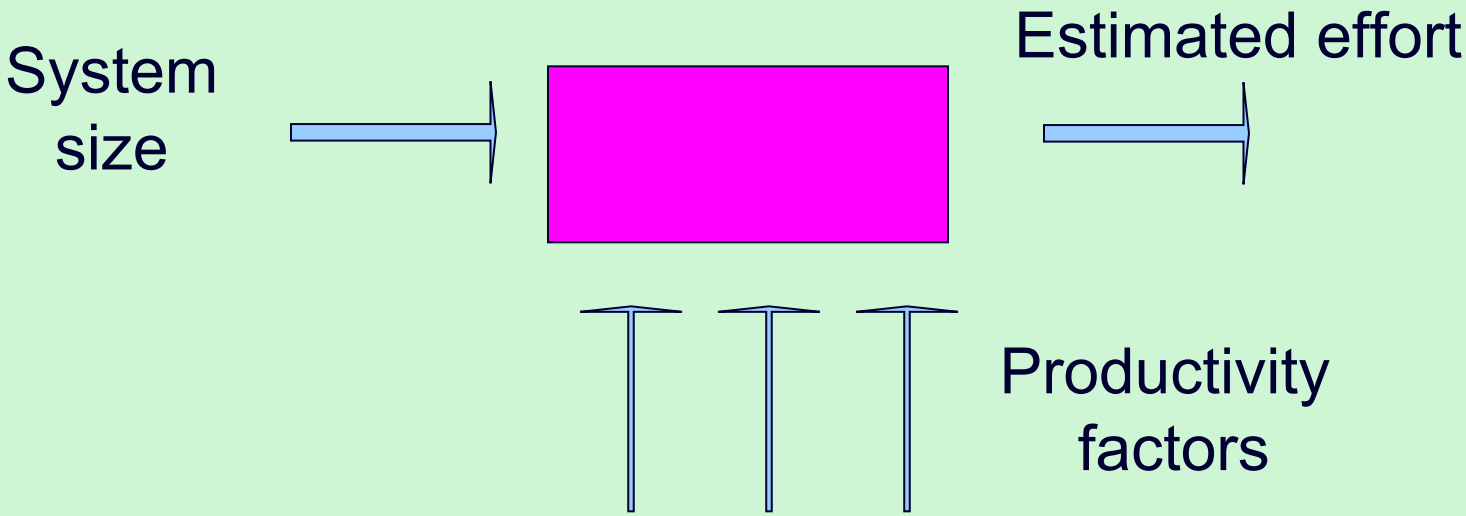
Parametric models

- Some models focus on task or system size e.g. Function Points
- FPs originally used to estimate Lines of Code, rather than effort



Parametric models

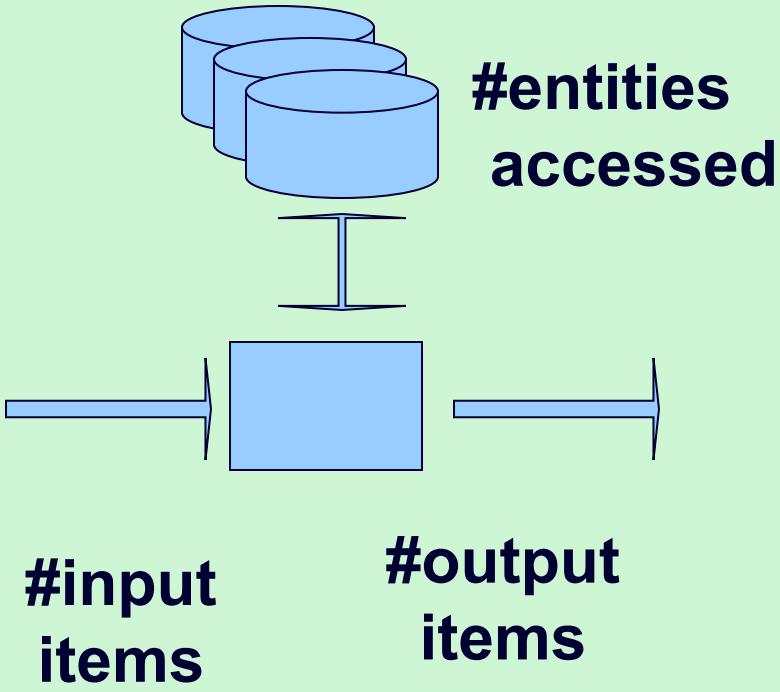
- Other models focus on productivity: e.g. COCOMO
- Lines of code (or FPs etc) an input



Function points Mark II

- Developed by Charles R. Symons
- 'Software sizing and estimating - Mk II FPA', Wiley & Sons, 1991.
- Builds on work by Albrecht
- Work originally for CCTA:
  - should be compatible with SSADM; mainly used in UK
- has developed in parallel to IFPUG FPs

Function points Mk II continued



For each transaction, count

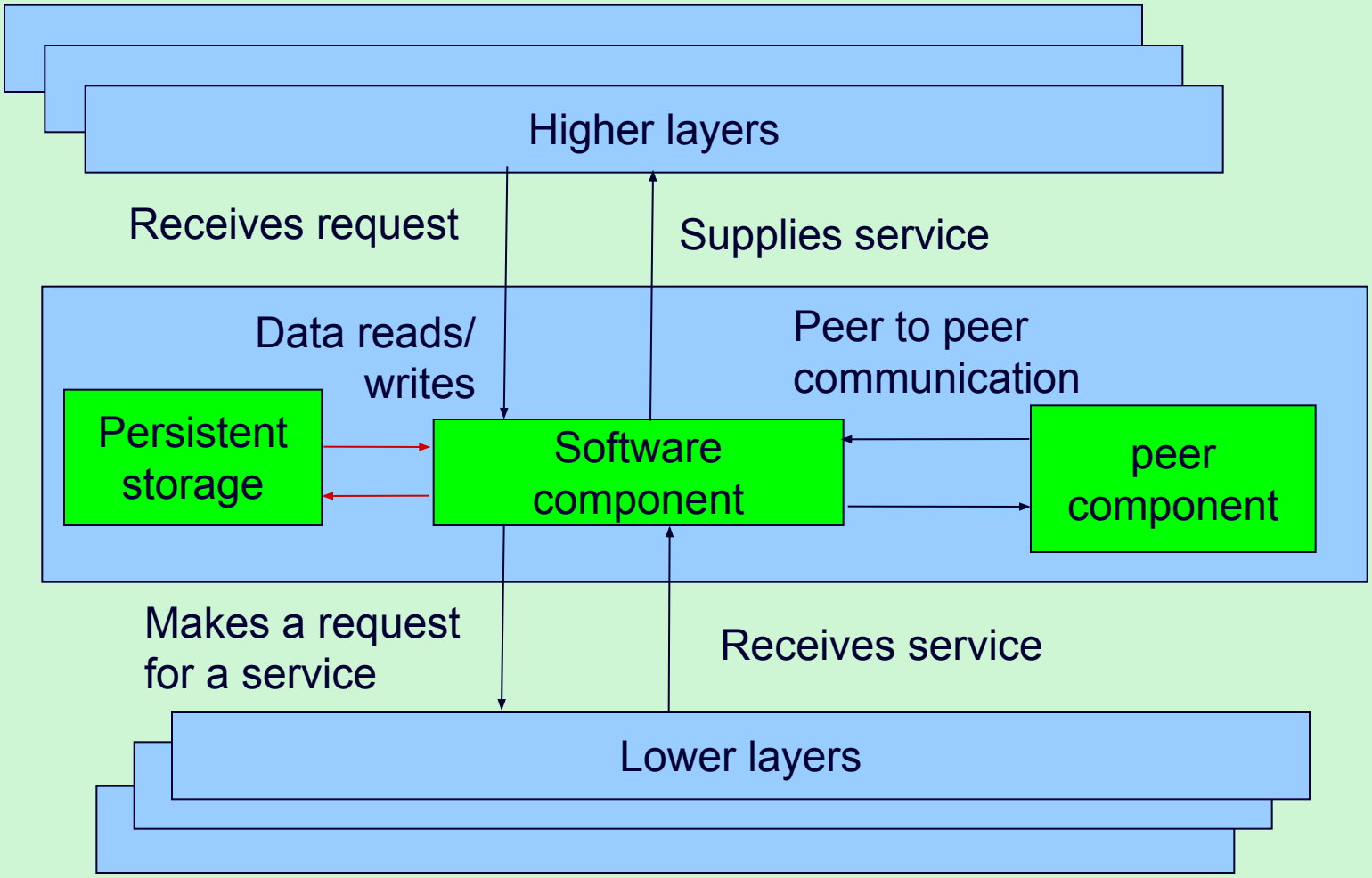
- data items input (N_i)
- data items output ($N_{o)}$
- entity types accessed (N_e)

FP count = N_i * 0.58 + N_e * 1.66 + N_o * 0.26

Function points for embedded systems

- Mark II function points, IFPUG function points were designed for information systems environments
- COSMIC FPs attempt to extend concept to embedded systems
- Embedded software seen as being in a particular 'layer' in the system
- Communicates with other layers and also other components at same level

Layered software



COSMIC FPs

The following are counted:

- Entries: movement of data into software component from a higher layer or a peer component
- Exits: movements of data out
- Reads: data movement from persistent storage
- Writes: data movement to persistent storage

Each counts as 1 'COSMIC functional size unit' (Cfsu)

COCOMO81

- Based on industry productivity standards - database is constantly updated
- Allows an organization to benchmark its software development productivity
- **Basic model**

  effort = c x size^{k}
- C and k depend on the type of system: organic, semi-detached, embedded
- Size is measured in 'kloc' ie. Thousands of lines of code

The COCOMO constants

System type	c	k
Organic (broadly, information systems)	2.4	1.05
Semi-detached	3.0	1.12
Embedded (broadly, real-time)	3.6	1.20

k exponentiation – 'to the power of…'
adds disproportionately more effort to the larger projects
takes account of bigger management overheads

Development effort multipliers (dem)

According to COCOMO, the major productivity drivers include:

Product attributes: required reliability, database size, product complexity

Computer attributes: execution time constraints, storage constraints, virtual machine (VM) volatility

Personnel attributes: analyst capability, application experience, VM experience, programming language experience

Project attributes: modern programming practices, software tools, schedule constraints

Using COCOMO development effort multipliers (dem)

An example: for analyst capability:

- Assess capability as very low, low, nominal, *high* or very high
- Extract multiplier:

very low	1.46
low	1.19
nominal	1.00
high	*0.80*
very high	0.71

- Adjust nominal estimate e.g. 32.6 x 0.80 = 26.8 staff months

Estimating by analogy

source cases

attribute values	effort
attribute values	effort
attribute values	effort
attribute values	effort
attribute values	effort
attribute values	effort

Use effort from source as estimate

target case

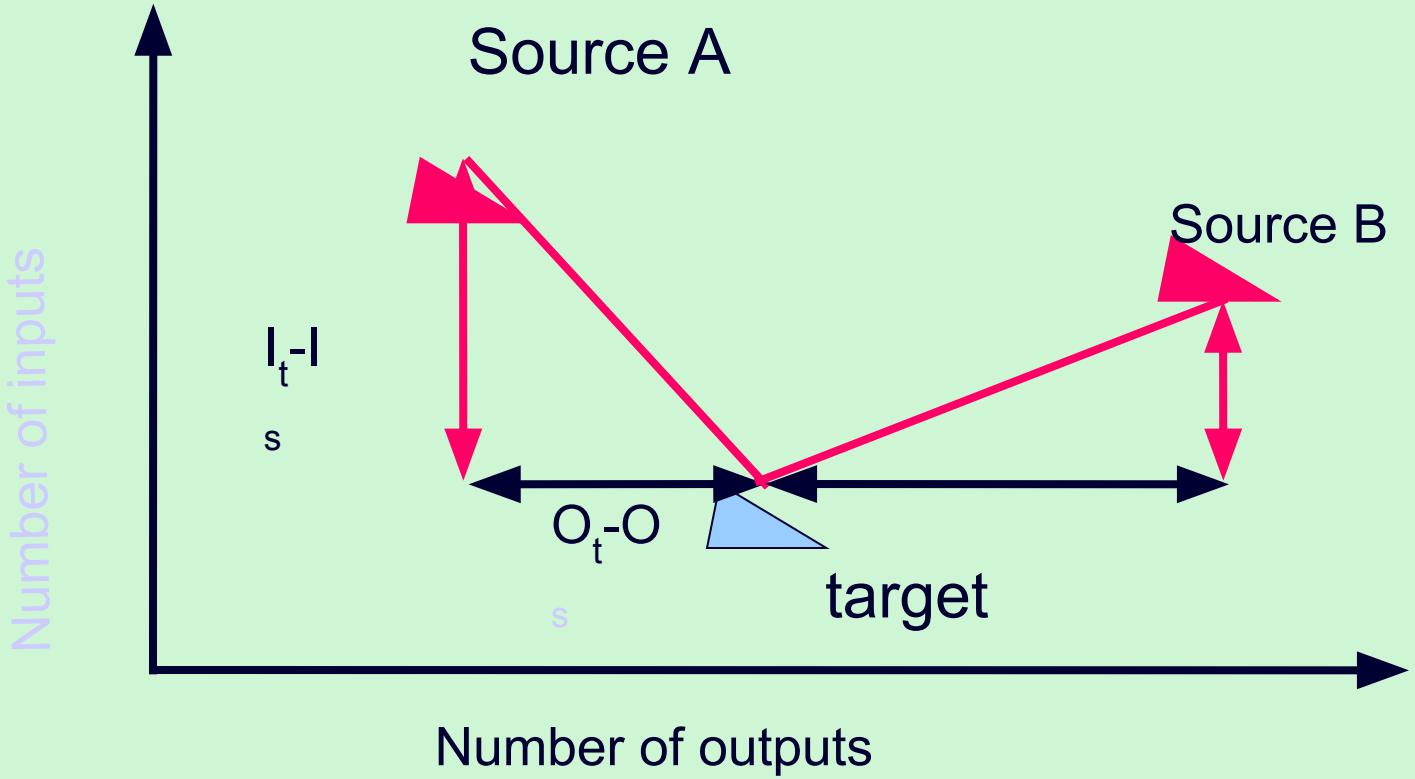
attribute values	?????

Select case with closet attribute values

Stages: identify

- Significant features of the current project
- previous project(s) with similar features
- differences between the current and previous projects
- possible reasons for error (risk)
- measures to reduce uncertainty

Machine assistance for source selection (ANGEL)



Euclidean distance = sq root $((I_t - I_s)^2 + (O_t - O_s)^2)$

Some conclusions: how to review estimates

Ask the following questions about an estimate

- What are the task size drivers?
- What productivity rates have been used?
- Is there an example of a previous project of about the same size?
- Are there examples of where the productivity rates used have actually been found?